Strategy Quant

سند مادر، منشور مهندسی، پروتکل اجرای عامل‌های هوش مصنوعی و ضمیمه کنترل کیفیت

نوع سند: مشخصات مرجع پروژه و قرارداد اجرایی عامل‌های هوش مصنوعی
وضعیت: نسخه بازنگری‌شده برای بررسی و تصویب
اصل حاکم: هیچ قابلیت، مرحله، نتیجه آزمایش یا ادعای تکمیل‌شده‌ای بدون شواهد قابل بررسی پذیرفته نمی‌شود .

بخش اول — سند مادر پروژه

۱. مأموریت و هدف نهایی

Strategy Quant یک پلتفرم مستقل پژوهش کمی، ساخت و تکامل استراتژی، آزمایش آماری و یادگیری ماشین برای بازارهای مالی است.

هدف، ساخت یک بک‌تستر ساده، ربات سیگنال‌دهی یا مجموعه‌ای از مدل‌های یادگیری ماشین نیست. هدف، توسعه یک سامانه پژوهشی قابل توسعه و قابل ممیزی است که بتواند داده بازار را دریافت کند، رفتار آن را تحلیل کند، استراتژی تولید کند، استراتژی‌ها را آزمایش کند، خطاها را شناسایی کند و نتایج را با شواهد قابل تکرار ارزیابی کند.

مهندسی معکوس رفتاری StrategyQuant Builder یکی از پایه‌های اصلی پروژه است؛ اما این کار باید با رعایت محدودیت‌های حقوقی، استفاده از شواهد مجاز و پیاده‌سازی مستقل انجام شود.

۱.۱. اهداف اصلی

1. بازسازی مستقل قابلیت‌های قابل مشاهده و مجاز StrategyQuant Builder.
2. ساخت مرکز دریافت، اعتبارسنجی و پژوهش داده‌های بازار.
3. ساخت موتور تولید و تکامل استراتژی.
4. ایجاد بک‌تست واقع‌گرایانه با مدل هزینه قابل تنظیم برای بروکرهای مختلف.
5. استفاده سنجیده از یادگیری ماشین و یادگیری عمیق.
6. ارزیابی آماری و کنترل بیش‌برازش.
7. ثبت آزمایش‌ها، نسخه داده‌ها، مدل‌ها، کدها و گزارش‌ها.
8. خودکارسازی کنترل کیفیت و تشخیص خطاهای عامل‌های هوش مصنوعی.
9. ایجاد رابط کاربری برای دریافت داده و مدیریت آزمایش‌ها.
10. آماده‌سازی مسیر توسعه برای MT5 و اجرای آزمایشی؛ با دروازه امنیتی مستقل برای هرگونه اجرای واقعی.
11. انتقال نهایی پروژه به مخزن عمومی موردنظر کاربر، فقط پس از تکمیل و تأیید و با رعایت امنیت و مجوزها.

۱.۲. اصول غیرقابل مذاکره

- صحت بر سرعت مقدم است.
- شواهد بر ادعا مقدم است.
- نتیجه منفی معتبر بهتر از نتیجه مثبت نامعتبر است.
- هیچ آزمونی نباید صرفاً توسط همان کدی که در حال ارزیابی است، تأیید شود.
- مدل پیچیده بدون مزیت اثباتشده نسبت به baseline پذیرفته نمی‌شود.
- خطای داده نباید به‌صورت پنهانی اصلاح شود.
- نتایج آزمون، بک‌تست، آموزش و اجرای زنده باید از یکدیگر تفکیک شوند.
- هیچ استراتژی صرفاً به دلیل سوددهی تاریخی تأیید نمی‌شود.
- هیچ نتیجه بک‌تستی به‌تنهایی مجوز معامله زنده نیست.
- هیچ عامل هوش مصنوعی اجازه ندارد برای سبزکردن آزمون، معیار پذیرش را پس از مشاهده نتیجه تغییر دهد.
- هیچ قابلیت اصلی پروژه به‌دلیل دشواری پیاده‌سازی حذف نمی‌شود؛ تغییر دامنه فقط با تصمیم ثبت‌شده مجاز است.

---

۲. محدوده پروژه و مرزهای مخزن

مخزن هدف برای انتقال نهایی:

`https://github.com/nafeZ1361/ketabkhane-ai-xauusd`

پوشه کاری محلی اعلام‌شده:

`strategy quant/`

بخشی از پروژه فعلی نیست و نباید تغییر کند `golden-librarytallaii` مخزن قدیمی.

۲.۱. محدودیت دسترسی

تا زمانی که فایل‌ها یا دسترسی واقعی در اختیار عامل قرار نگرفته‌اند، عامل نباید ادعا کند که مخزن، لپ‌تاپ، فایل‌ها یا محیط اجرایی کاربر را بررسی کرده است.

انجام نمی‌شود GitHub تا قبل از اجازه صریح کاربر، هیچ تغییری در.

در مرحله انتقال نهایی، انتشار عمومی فقط پس از بررسی مجوزها، حذف اسرار، بررسی داده‌های دارای محدودیت انتشار و تأیید کاربر انجام خواهد شد.

۲.۲. مرز شواهد

هر گزارش باید وضعیت ادعا را مشخص کند:

- `VERIFIED`: با مدرک قابل بررسی تأیید شده است.
- `FAILED`: آزمون اجرا شده و شکست خورده است.

- `BLOCKED`: اجرای آزمون به مانع مشخص برخورد کرده است.
- `UNKNOWN`: شواهد کافی وجود ندارد.
- `PROPOSED`: پیشنهاد شده اما اجرا یا تأیید نشده است.
- `NOT_RUN`: آزمون هنوز اجرا نشده است.
- `WAIVED`: الزام با تصمیم مستند و دارای دامنه مشخص معاف شده است.

وضعیت `WAIVED` به معنی موفقیت آزمون نیست.

---

۳. معماری کلان

معماری به ماژول‌های مستقل تقسیم می‌شود. زبان برنامه‌نویسی، چارچوب رابط کاربری، پایگاه داده و کتابخانه‌های اصلی باید در مرحله طراحی فنی و پس از بررسی شواهد انتخاب شوند؛ این سند آن تصمیم‌ها را از پیش تحمیل نمی‌کند.

۳.۱. ماژول‌های اصلی

الف. Data & Research Center

- انتخاب یا دریافت پوشه فایل‌های CSV و فایل ZIP.
- شناسایی فایل‌ها، ستون‌ها، فرمت‌ها و تایم‌فریم‌ها.
- شناسایی نماد و منبع داده.
- اعتبارسنجی و ممیزی داده.
- ثبت اصلاحات مجاز و گزارش مشکلات.
- مدیریت نسخه و منشأ داده.

ب. Data Quality & Normalization

- بررسی ترتیب زمانی.
- تشخیص زمان‌های تکراری.
- بررسی مقادیر گمشده.
- بررسی OHLC.
- تشخیص شکاف‌های زمانی.
- تشخیص ناهماهنگی تایم‌فریم.
- تشخیص قیمت نامعتبر.
- تشخیص داده‌های ناقص یا ناسازگار.
- نرمال‌سازی با حفظ اطلاعات اصلی.
- جلوگیری از اصلاح خاموش یا حذف داده بدون گزارش.

ج. Feature & Label Engine

- تولید ویژگی‌های تکنیکال و آماری.
- تولید برچسب‌های پژوهشی.
- کنترل دسترسی زمانی ویژگی‌ها.

- جلوگیری از نشت اطلاعات.
- ثبت وابستگی و نسخه هر ویژگی.
- امکان بازتولید مجموعه ویژگی‌ها.

د. Strategy DSL & Builder

- تعریف زبان یا ساختار استاندارد استراتژی.
- تعریف قواعد ورود، خروج، توقف، اندازه موقعیت و مدیریت ریسک.
- اعتبارسنجی ساختار استراتژی.
- تبدیل استراتژی به نمایش قابل اجرا.
- ثبت نسخه و پارامترهای استراتژی.

هـ. Strategy Generation & Evolution

- تولید استراتژی‌های قاعده‌محور.
- جست‌وجوی فضای پارامترها.
- تکامل و ترکیب استراتژی‌ها.
- حذف استراتژی‌های نامعتبر.
- کنترل پیچیدگی و تکرار.
- ثبت کامل فرایند انتخاب و حذف.

تنها پس از ارزیابی تناسب و هزینه نگهداری انتخاب Optuna و pymoo، DEAP، الگوریتم‌هایی مانند الگوریتم ژنتیک می‌شوند.

و. Backtesting Engine

- اجرای معاملات بر مبنای زمان.
- شبیه‌سازی سفارش و موقعیت.
- محاسبه سود و زیان.
- اعمال هزینه‌های معاملاتی.
- کنترل استفاده از اطلاعات آینده.
- ثبت معاملات و رویدادهای اجرا.
- محاسبه معیارهای عملکرد و ریسک.

ز. Statistical Validation

- جداسازی داده آموزش، اعتبارسنجی و آزمون.
- آزمون خارج از نمونه.
- Walk-Forward.
- Monte Carlo.
- تحلیل حساسیت.
- آزمون پایداری.
- کنترل چندبارآزمایی و انتخاب بهترین نتیجه از میان تعداد زیادی آزمایش.

ح. ML Research Engine

مدل‌های قابل بررسی، بسته به نیاز واقعی پروژه:

- مدل‌های خطی و رگرسیون.
- Decision Tree.
- Random Forest.
- XGBoost و LightGBM.
- مدل‌های عصبی ساده.
- PyTorch.
- GRU و LSTM.
- مدل‌های تشخیص رژیم بازار و خوشه‌بندی.
- یا مدل‌های پیچیده‌تر، تنها در صورت توجیه تجربی Transformer.

هیچ مدل خاصی از پیش برنده فرض نمی‌شود. مدل پیچیده باید با baseline مناسب مقایسه شود.

ط. Experiment Registry & Reproducibility

- ثبت آزمایش‌ها.
- ثبت تنظیمات و seed.
- ثبت هش داده و کد.
- ثبت تقسیم داده.
- ثبت معیارهای ارزیابی.
- ثبت محیط اجرا.
- ثبت گزارش‌ها و artifactها.
- امکان بررسی و بازتولید نتایج.

ی. User Interface

- دریافت CSV یا ZIP.
- انتخاب پوشه داده.
- نمایش وضعیت اعتبارسنجی.
- نمایش مشکلات و اصلاحات.
- نمایش استراتژی‌ها و نتایج.
- مدیریت آزمایش‌ها.
- ارائه گزارش قابل بررسی.
- نمایش صریح خطا و علت توقف.

ک. MT5 & Execution Integration

- در مراحل مجاز MT5 پشتیبانی از اتصال و آزمایش.
- بررسی مشخصات واقعی نماد و بروکر.
- آزمایش دمو.
- کنترل ریسک و محدودیت سفارش.
- توقف اضطراری.
- دروازه مستقل برای فعال‌سازی معاملات واقعی.

---

۴. مهندسی معکوس رفتاری StrategyQuant Builder

این فعالیت باید رفتارهای قابل مشاهده را بررسی کند، نه اینکه از روی نام قابلیت‌ها یا حدس درباره پیاده‌سازی داخلی، نتیجه‌گیری شود.

۴.۱. محدوده

برای هر قابلیت مورد بررسی باید مشخص شود:

- رفتار ورودی و خروجی.
- پارامترهای قابل مشاهده.
- حالت‌های مرزی.
- خطاهای احتمالی.
- وابستگی به نسخه.
- رفتار تصادفی یا غیرقطعی.
- محدودیت‌های مجوز و استفاده.
- سطح اطمینان شواهد.

۴.۲. تفکیک سه نوع هدف

1. **رفتار سازگار:** خروجی مستقل در محدوده معیار از پیش تعریف‌شده با مرجع قابل مقایسه است.
2. **بازسازی مستقل:** قابلیت مشابه با طراحی و کد مستقل ساخته شده است، اما برابری رفتاری هنوز اثبات نشده است.
3. **قابلیت پژوهشی جدید:** قابلیت مستقلی است که برای نیاز پروژه ساخته می‌شود و لزوماً معادل SQ نیست.

این سه حالت نباید با یکدیگر اشتباه گرفته شوند.

۴.۳. شرط حقوقی

پیش از شروع آزمایش‌هایی که به مهندسی معکوس SQ ،وابسته‌اند، باید بررسی حقوقی مجوز نرم‌افزار EULA و قوانین قابل اعمال انجام شود.

تا زمانی که مجازبودن روش موردنظر روشن نشده است:

- تحلیل مستندات عمومی و شواهد مجاز قابل بررسی است.
- آزمایش یا استخراجی که احتمال نقض مجوز دارد، متوقف می‌ماند.
- هیچ کد، داده داخلی، فایل اجرایی یا دارایی اختصاصی بدون مجوز وارد پروژه نمی‌شود.
- هر ابهام مهم با تصمیم حقوقی ثبت‌شده تعیین تکلیف می‌شود.

این سند جایگزین مشاوره حقوقی نیست.

---

۵. مرکز داده و مشخصات بروکر

۵.۱. ورودی

را داشته باشد. در نسخه‌های توسعه بعدی، دریافت پوشه شامل چند فایل و چند تایم‌فریم نیز ZIP و CSV سامانه باید امکان دریافت باید پشتیبانی شود.

۵.۲. تشخیص نماد

نام نماد نباید ثابت فرض شود.

برای تشخیص نماد استفاده کند alias سامانه باید از اطلاعات فایل، نام ستون‌ها، فراداده، مشخصات بروکر و جدول.

نام نماد موجود در فایل با نام نماد قابل معامله در بروکر یکسان فرض نمی‌شود.

اگر چند تطبیق معتبر وجود داشته باشد یا شواهد کافی نباشد، سامانه باید از کاربر تأیید بگیرد و از نگاشت حدسی خودداری کند.

۵.۳. بروکر

بروکر فعلی موردنظر کاربر است، اما نباید تنها بروکر پشتیبانی‌شده در طراحی Aron Group .پروفایل بروکر باید قابل تنظیم باشد باشد.

مشخصات زیر باید از شواهد معتبر و مربوط به نماد و حساب به دست آیند:

- Spread.
- Commission.
- Slippage.
- Swap.
- Contract size.
- Tick size و tick value.
- حداقل حجم و گام حجم.
- ساعات معامله.
- محدودیت‌های سفارش.
- مارجین و محدودیت‌های اجرایی.

مقدار نامعلوم نباید به‌عنوان مقدار واقعی بروکر استفاده شود.

اگر مشخصات ضروری برای نتیجه معتبر موجود نباشد، بک‌تست باید با وضعیت محدودیت مشخص شود یا متوقف بماند.

۵.۴. زمان

منطقه زمانی مرجع کاربر `Asia/Tehran` است.

را داشته باشد. منطقه زمانی مبدأ، منبع زمان و UTC باشد و امکان نرمال‌سازی به timezone-aware زمان داخلی پردازش باید هرگونه تبدیل باید ثبت شود.

ثابت و دست‌ساز مدیریت شود offset اختلاف ساعت تابستانی نباید با یک.

---

۶. سیاست داده و اصلاح خطا

.هر فایل پیش از استفاده در آموزش یا بک‌تست باید ممیزی شود

۶.۱. خطاهای قابل اصلاح

.اصلاح فقط وقتی مجاز است که قاعده آن مشخص، اثر آن قابل ارزیابی و سابقه آن ثبت‌شده باشد

:برای هر اصلاح باید ثبت شود

- .مقدار یا وضعیت اولیه
- .مقدار یا وضعیت جدید
- .علت اصلاح
- .قاعده اعمال‌شده
- .تعداد ردیف‌های متأثر
- .اثر بر مجموعه داده
- .نتیجه آزمون پس از اصلاح

۶.۲. خطاهای نیازمند توقف

ناسازگار، زمان‌های مبهم، نگاشت نامعتبر نماد یا فقدان داده ضروری، OHLC ،در صورت وجود ابهام درباره ترتیب واقعی بازار .سامانه نباید بدون اعلام خطا ادامه دهد

۶.۳. حفظ منبع اصلی

.داده اصلی نباید با نسخه اصلاح‌شده جایگزین غیرقابل بازگشت شود

.نسخه خام، نسخه نرمال‌شده، گزارش ممیزی و شناسه داده باید قابل تفکیک باشند

۶.۴. دیتاست اصلی پژوهش

که پیش‌تر به پروژه معرفی شده، یکی از ورودی‌های موردنظر است. گزارش تاریخی از وجود XAUUSD M1 دیتاست تاریخی فایلی با حدود ۳٫۴۹ میلیون ردیف و بازه‌ای از ۲۰۱۶ تا ۲۰۲۶ وجود دارد؛ اما تا زمان دسترسی واقعی، بررسی هش و ممیزی .مجدد، این اطلاعات نباید به‌عنوان وضعیت فعلی فایل تأییدشده تلقی شوند

---

۷. استراتژی، بک‌تست و ارزیابی آماری

۷.۱. بک‌تست پایه

.موتور باید بر اساس اطلاعات موجود در زمان تصمیم، معاملات را شبیه‌سازی کند

قواعد ورود و خروج، زمان اجرا، قیمت اجرا و نحوه محاسبه سود و زیان باید مشخص باشند.

۷.۲. مدل هزینه

مدل هزینه باید قابل تنظیم و وابسته به مشخصات نماد، بروکر و شرایط حساب باشد.

بک‌تست بدون هزینه فقط برای توسعه اولیه یا تحلیل مشخص مجاز است و باید با برچسب `NO_COST_RESEARCH_ONLY` مشخص شود.

نتایج بدون هزینه نباید با نتایج دارای هزینه به‌عنوان یک آزمایش یکسان مقایسه شوند.

۷.۳. معیارهای ارزیابی

حداقل مجموعه معیارها باید بر اساس نوع استراتژی شامل موارد زیر باشد:

- تعداد معاملات.
- سود و زیان خالص.
- Profit Factor.
- Drawdown.
- Sharpe و Sortino.
- Expectancy.
- نرخ برد، با توجه به محدودیت‌های آن.
- توزیع سود و زیان معاملات.
- مدت نگهداری.
- هزینه معاملات.
- پایداری در بازه‌های مختلف.
- عملکرد خارج از نمونه.
- حساسیت به تغییر پارامترها.

۷.۴. کنترل بیش‌برازش

- تقسیم زمانی داده باید مشخص باشد.
- داده آینده نباید در ویژگی‌ها یا تصمیم‌های گذشته استفاده شود.
- انتخاب مدل و پارامتر نباید به‌صورت پنهانی از داده آزمون نهایی استفاده کند.
- تعداد آزمایش‌های انجام‌شده باید ثبت شود.
- تعداد استراتژی‌های بررسی‌شده و روش انتخاب برنده باید در گزارش نهایی ذکر شود.
- برای ارزیابی نتایج منتخب از میان تعداد زیادی آزمایش، اصلاح چندبارآزمایی مانند Deflated Sharpe Ratio یا روش آماری مناسب دیگر باید بررسی و در صورت تناسب اعمال شود.
- نتیجه خام Sharpe به‌تنهایی معیار کافی برای پذیرش نیست.

---

۸. سیاست یادگیری ماشین و PyTorch

ML ابزار پژوهش است، نه تضمین سودآوری.

۸.۱. ترتیب توسعه

1. تعریف مسئله و هدف پیش‌بینی.
2. تعریف baseline ساده.
3. تعیین تقسیم زمانی داده.
4. تعریف ویژگی‌ها و برچسب‌ها.
5. کنترل leakage.
6. آموزش مدل.
7. ارزیابی خارج از نمونه.
8. مقایسه با baseline.
9. تحلیل خطا.
10. تصمیم درباره نگهداری یا حذف مدل.

۸.۲. مدل‌های پیچیده

افزودن LSTM، GRU، Transformer، CNN یا RL فقط در صورتی مجاز است که مسئله، داده و معیار ارزیابی توجیه کنند.

پیچیدگی بیشتر بدون بهبود قابل اثبات، دلیل پذیرش نیست.

۸.۳. بازتولیدپذیری

هر آزمایش ML باید تا حد امکان موارد زیر را ثبت کند:

- شناسه و هش داده.
- نسخه کد.
- تقسیم داده.
- seed.
- نسخه Python و کتابخانه‌ها.
- پیکربندی مدل.
- پیکربندی آموزش.
- معیارها.
- خروجی مدل.
- هش artifactها.
- محدودیت‌های بازتولیدپذیری محیطی.

مقایسه دو اجرای مستقل باید بخشی از آزمون‌های حیاتی باشد.

---

۹. دانش پروژه و مدیریت زمینه

ساختار آن با مخزن ،Obsidian نگهداری شود و در صورت استفاده از Markdown مرجع وضعیت پروژه باید در اسناد مستندات سازگار باشد.

ساختار پیشنهادی:

- `00-Home`
- `01-Project-Map`
- `02-Architecture`
- `03-Research`
- `04-Decisions`
- `05-Stages`
- `06-Experiments`
- `07-Bugs-and-Incidents`
- `08-Knowledge-Base`
- `09-Sources`
- `10-Reviews`
- `11-Next-Actions`
- `Templates`

اسناد وضعیت، تصمیم‌ها و آزمایش‌ها باید در قالب استاندارد ثبت شوند. هر عامل جدید باید بتواند بدون اتکا به حافظه مکالمه، وضعیت واقعی پروژه را از اسناد و artifactهای موجود بازیابی کند.

---

۱۰. مراحل کلان پروژه

مرحله ۰۰ — قواعد، دسترسی و محدوده

تعیین محدوده، دسترسی واقعی، قواعد امنیتی، وضعیت مخزن و شیوه ثبت شواهد.

مرحله ۰۱ — دروازه حقوقی و منابع

بررسی شرایط مجاز پژوهش، مستندات قابل استفاده و محدودیت‌های انتشار.

مرحله ۰۲ — شناخت رفتار و Oracle

تعریف قابلیت‌های موردنظر، مرجع مستقل و پروتکل کشف رفتار SQ.

مرحله ۰۳ — معماری و قراردادهای ماژول

تعریف APIها، مرزها، وابستگی‌ها و معیار پذیرش.

کوچک MVP — مرحله ۰۴

تا بک‌تست ساده و خروجی معاملات CSV اجرای مسیر حداقلی.

مرحله ۰۵ — اعتبارسنجی کامل داده

گسترش ممیزی، نرمال‌سازی، چند فایل و چند تایم‌فریم.

Feature & Label Engine — مرحله ۰۶

تولید ویژگی‌ها و برچسب‌ها همراه با آزمون‌های leakage.

مرحله ۰۷ — بک‌تست پایه

اعتبارسنجی قواعد معامله، زمان‌بندی و ثبت معاملات.

مرحله ۰۸ — مدل هزینه

افزودن هزینه‌های بروکر و آزمون سازگاری مدل هزینه.

Strategy DSL & Builder — مرحله ۰۹

ساخت زبان و موتور اجرای استراتژی.

مرحله ۱۰ — تولید و تکامل استراتژی

افزودن تولید، جست‌وجو و تکامل با کنترل پیچیدگی.

مرحله ۱۱ — ارزیابی آماری

و کنترل بیش‌برازش Monte Carlo، Walk-Forward، آزمون خارج از نمونه.

پایه ML — مرحله ۱۲

مقایسه مدل‌های ساده و تعیین ارزش واقعی ML.

و مدل‌های پیشرفته PyTorch — مرحله ۱۳

فقط در صورت توجیه و با معیار پذیرش مستقل.

مرحله ۱۴ — رجیستری و بازتولیدپذیری

ثبت آزمایش‌ها، هش‌ها، امضاها و آزمون‌های تکرارپذیری.

مرحله ۱۵ — رابط کاربری

ایجاد مسیر کاربر برای دریافت داده، اجرای پژوهش و مشاهده خطاها.

مرحله ۱۶ — کنترل خودکار و پایش عامل‌ها

اجرای اسکریپت‌های کنترل کیفیت، پایش و بررسی ضدتقلب.

مرحله ۱۷ — امنیت و عملکرد

بررسی وابستگی‌ها، ورودی‌ها، مجوزها، مصرف منابع و امنیت artifactها.

مرحله ۱۸ — بسته‌بندی و انتقال

آماده‌سازی بسته قابل اجرا، مستندات نصب و انتقال نهایی به مخزن مورد تأیید کاربر.

مرحله ۱۹ — پذیرش پژوهشی

بررسی نهایی اعتبار آماری و شواهد عملکرد.

مرحله ۲۰ — اجرای آزمایشی MT5

آزمون دمو، سازگاری نماد، هزینه و ایمنی اجرا.

مرحله ۲۱ — دروازه مستقل اجرای واقعی

فقط پس از مجوز صریح کاربر و تأیید تمام الزامات امنیتی و پژوهشی.

عبور از یک مرحله، مراحل بعدی را به‌صورت خودکار مجاز نمی‌کند.

بخش دوم — ضمیمه اجرایی و حاکمیتی عامل‌های هوش مصنوعی

۱۱. پروتکل استاندارد ساخت هر ماژول

هر ماژول باید با یک چرخه مشخص ساخته شود. عامل مجاز نیست مستقیماً از توضیح کلی مسئله به پیاده‌سازی گسترده برود.

گام‌های اجباری

۱. تعریف مسئله و خروجی مورد انتظار.

۲. تعریف API و قراردادهای ورودی و خروجی.

۳. تعیین Oracle یا توضیح مکتوب علت نبود آن.

۴. تعیین baseline و معیار پذیرش پیش از اجرای آزمون اصلی.

۵. طراحی تست‌های مثبت، منفی و مرزی.

۶. طراحی تست مستقل از پیاده‌سازی.

۷. پیاده‌سازی کوچک‌ترین تغییر لازم.

۸. اجرای آزمون‌های مرتبط.

۹. اجرای بررسی مستقل.

۱۰. ثبت artifactها، نتایج و محدودیت‌ها.

۱۱. تصمیم رسمی درباره پذیرش یا رد.

۱۲. ثبت تحویل برای عامل یا مرحله بعد.

قالب تحویل ماژول

هر ماژول باید فایل `MODULE_SUMMARY.md` شامل، حداکثر یک صفحه، داشته باشد؛ حداکثر یک صفحه، شامل:

- هدف.
- API عمومی.
- وابستگی‌ها.
- ورودی و خروجی.
- روش اجرا.
- تست‌ها.
- محدودیت‌ها.
- Oracle مرتبط.
- وضعیت تأیید.
- شناسه آخرین بررسی.
- مشکلات باز.

ماژول بدون قرارداد، تست و خلاصه قابل انتقال، برای ادغام آماده نیست.

---

۱۲. Golden Dataset کشف رفتار و Oracle،

۱۲.۱. Oracle مستقل

برای هر قابلیت مهم باید یک Oracle مستقل تعریف شود.

Oracle می‌تواند یکی از این موارد باشد:

- خروجی مرجع مجاز و قابل ثبت.
- محاسبه دستی مستقل.
- پیاده‌سازی مرجع کوچک و جداگانه.
- داده استاندارد با نتیجه معلوم.
- ویژگی رفتاری قابل مشاهده که با روش مستقل قابل بررسی است.

کد تولیدی که در حال آزمایش است، نباید تنها مرجع تأیید صحت خودش باشد.

۱۲.۲. Oracle ماتریس

برای هر قابلیت، فیلدهای زیر الزامی‌اند:

فیلد	توضیح
`capability_id`	شناسه یکتای قابلیت
`behavior_claim`	رفتار مورد انتظار
`oracle_type`	نوع مرجع
`oracle_source`	منبع یا روش تولید مرجع
`version`	نسخه مرجع
`input_hash`	هش ورودی آزمون
`expected_output_hash`	هش خروجی مورد انتظار، در صورت کاربرد
`tolerance`	تلورانس مجاز
`independence_check`	روش اثبات استقلال
`evidence_path`	مسیر شواهد
`status`	وضعیت پذیرش
`limitations`	محدودیت‌ها

قابلیت بدون Oracle می‌تواند در وضعیت آزمایشی توسعه یابد، اما نباید به‌عنوان رفتار تأییدشده SQ معرفی شود.

۱۲.۳. کشف رفتار SQ

برای هر قابلیت مورد بررسی:

۱. ثبت نام قابلیت و نسخه دقیق SQ.

۲. ثبت محیط و تنظیمات.

۳. ثبت ورودی و پیش‌شرط‌ها.

۴. تعریف فرضیه رفتاری.

۵. انتخاب آزمایش کمینه.

۶. اجرای آزمایش فقط در محدوده مجاز.

۷. ثبت خروجی اصلی و خطاها.

۸. تکرار آزمایش برای بررسی پایداری.

۹. بررسی حالت‌های مرزی و رفتارهای متفاوت.

۱۰. ثبت نتیجه، سطح اطمینان و محدودیت‌ها.

رفتار مشاهده‌نشده نباید به‌عنوان رفتار قطعی SQ ثبت شود.

۱۲.۴. Golden Dataset

برای هر قابلیت مهم باید مجموعه داده مرجع متناسب با همان قابلیت ایجاد شود.

طرح اولیه ۳×۳×۳، در صورت تناسب، می‌تواند برای پوشش سه سطح از سه عامل استفاده شود؛ اما تعداد نمونه‌ها به‌تنهایی تضمین‌کننده پوشش نیست.

مجموعه مرجع باید شامل موارد زیر باشد:

- حالت عادی.
- حالت مرزی.
- ورودی نامعتبر.
- داده کوچک برای بررسی دستی.
- داده دارای زمان‌های حساس.
- داده دارای هزینه‌های مشخص، در صورت ارتباط.
- خروجی مرجع مستقل.
- نسخه و هش ورودی و خروجی.

افزودن نمونه‌ها باید بر اساس خطاهای کشف‌شده، تحلیل پوشش و ریسک انجام شود.

---

۱۳. کنترل کیفیت، Anti-Gaming و آستانه‌های پذیرش

۱۳.۱. اصل ضدتقلب

عامل یا توسعه‌دهنده نباید با تغییر معیار، تضعیف تست، حذف شکست یا هماهنگ‌کردن خطای کد و تست، نتیجه مثبت مصنوعی ایجاد کند.

موارد زیر تخلف محسوب می‌شوند:

- حذف assertion بدون توجیه و تأیید.
- کاهش سخت‌گیری آزمون برای سبزکردن build.
- تغییر کد و تست برای تأیید یک خطای مشترک.
- حذف تست شکست‌خورده بدون ثبت دلیل.
- نادیده‌گرفتن خطاهای شناخته‌شده.
- تغییر آستانه پس از مشاهده نتیجه.
- استفاده از داده آزمون نهایی برای تنظیم مدل.
- ارائه خروجی ساختگی به‌عنوان خروجی واقعی.
- اعلام اجرای تستی که اجرا نشده است.
- تغییر هش یا گزارش برای پنهان‌کردن تفاوت.
- ادغام تغییرات بدون بررسی مستقل در بخش‌های حیاتی.

۱۳.۲. روش‌های تشخیص

برای کشف این موارد، بررسی کد به‌تنهایی کافی نیست. کنترل باید ترکیبی باشد از:

- بررسی diff کد و تست.
- اجرای مجدد تست‌ها در محیط مستقل.
- بررسی حذف یا تغییر assertion.
- بررسی تست‌های غیرفعال یا skip شده.
- اجرای mutation testing.
- مقایسه گزارش‌های قبلی و جدید.
- بررسی پوشش شاخه‌ها و مسیرهای حیاتی.
- آزمون منفی برای اطمینان از اینکه کنترل واقعاً شکست را تشخیص می‌دهد.
- بازبینی مستقل تغییرات مرتبط با معیار پذیرش.

۱۳.۳. آستانه‌های اولیه

اعداد زیر پیش‌فرض‌های اولیه برای طراحی کنترل کیفیت‌اند. پیش از استفاده نهایی باید با ساختار واقعی کد و ریسک هر ماژول تطبیق داده شوند و در ADR ثبت شوند.

شاخص	آستانه اولیه	اقدام در صورت شکست
خطای بحرانی باز در مسیر پذیرش	۰	توقف پذیرش

توقف مرحله	موفق ٪۱۰۰	تست‌های Critical Path
تکمیل تست یا ثبت مانع	٪حداقل ۹۰	پوشش شاخه‌ای مسیرهای حیاتی
بررسی تست‌های ناکافی	٪حداقل ۷۰	ماژول حیاتی Mutation Score
بررسی و تصمیم ثبت‌شده	٪حداقل ۶۰	سایر ماژول‌های Mutation Score حساس
رد تغییر	۰	تست‌های حذف یا غیرفعال‌شده بدون مجوز
توقف پذیرش	مطابق معیار از پیش تعیین‌شده ٪۱۰۰	آزمون‌های تکرارپذیری حیاتی
ناقص artifact رد	٪۱۰۰	های حیاتی با هش معتبرartifact
توقف انتشار	۰	حل‌نشده Critical یافته امنیتی
تغییر وضعیت به آزمایشی	۰	مستقل Oracle قابلیت‌های تأییدشده بدون
رد گزارش مرحله	۰	مسائل بدون شناسه در گزارش توقف

بالا به‌تنهایی اثبات نمی‌کند که تست‌ها از نظر معنایی Mutation Score ،این آستانه‌ها تضمین‌کننده کیفیت نیستند. برای مثال درست‌اند.

۱۳.۴. نسبت تست به کد

نسبت ساده تعداد خطوط تست به خطوط تولیدی، شاخص کیفیت قابل اتکایی نیست؛ زیرا به زبان و سبک پروژه وابسته است.

بنابراین، این نسبت فقط به‌عنوان شاخص تشخیصی گزارش می‌شود و معیار پذیرش مستقل نیست. معیار اصلی، پوشش مسیرهای حیاتی، کیفیت assertionها، mutation testing و شواهد Oracle است.

۱۳.۵. پایش خودکار

اسکریپت زیر به‌عنوان ابزار الزامی مرحله پایش تعریف می‌شود:

`scripts/monitor_agents.py`

وظایف مورد انتظار:

- خواندن شناسه مسئله‌ها و تلاش‌ها.
- بررسی کامل‌بودن artifactهای هر مرحله.
- استخراج وضعیت تست‌ها.
- بررسی هش‌ها و امضاهای موجود.
- شمارش تست‌های حذف‌شده یا غیرفعال‌شده در diff، در صورت وجود داده لازم.
- خواندن گزارش mutation testing، در صورت فعال‌بودن آن.
- محاسبه شاخص‌های پایش.

- تولید گزارش ماشین‌خوان و Markdown.
- اعلام خطا در صورت فقدان داده موردنیاز.

این ابزار باید در هر دروازه پذیرش اجرا شود. تا پیش از پیاده‌سازی و آزمون واقعی آن، وضعیت آن PROPOSED است، نه VERIFIED.

---

۱۴. زنجیره اعتماد، هش و امضای artifactها

۱۴.۱. اصل اعتماد

هش، یکپارچگی بایت‌ها را بررسی می‌کند؛ اما به‌تنهایی صحت علمی نتیجه، هویت تولیدکننده یا استقلال آزمون را ثابت نمی‌کند.

بنابراین، زنجیره اعتماد باید چند لایه داشته باشد:

۱. هش artifactها.

۲. ثبت نسخه کد و محیط.

۳. ثبت شناسه اجرا.

۴. ثبت ورودی‌ها و خروجی‌ها.

۵. امضای artifactهای حساس.

۶. بررسی امضا توسط مصرف‌کننده مستقل.

۷. ثبت زنجیره تحویل و پذیرش.

۱۴.۲. امضای پایه

از MVP، artifactهای حیاتی باید علاوه بر هش، امضای قابل بررسی داشته باشند.

گزارش عمومی، فایل داده یا بسته انتشار قرار Git، با کلید محلی خارج از مخزن است. کلید نباید در HMAC، یک گزینه اولیه گیرد.

برای اثبات مستقل هویت چند عامل کافی نیست، زیرا هر دارنده کلید می‌تواند امضا تولید کند. برای اعتماد میان HMAC، بااین‌حال عامل‌های مستقل یا محیط‌های جداگانه، سازوکار امضای نامتقارن و مدیریت هویت باید طراحی و تصویب شود.

۱۴.۳. artifactهای حساس

- گزارش آزمون.
- manifest آزمایش.
- نتایج پذیرش.

- گزارش ممیزی داده.
- گزارش امنیتی.
- گزارش مقایسه با SQ.
- خروجی نهایی پژوهش.
- گزارش انتشار.

۱۴.۴. بررسی مستقل

مصرف‌کننده artifact باید بتواند:

- هش را دوباره محاسبه کند.
- امضا را اعتبارسنجی کند.
- هویت یا کلید مورد اعتماد را بررسی کند.
- نسخه و زمان اجرای گزارش‌شده را تطبیق دهد.
- مشخص کند آیا artifact پس از امضا تغییر کرده است.

امضای معتبر، جایگزین اجرای مستقل آزمون نیست.

---

۱۵. مدیریت مسئله، تلاش‌ها و توقف

۱۵.۱. شناسه یکتای مسئله

هر مشکل قابل پیگیری باید در `OPEN_ISSUES.md` با شناسه یکتا ثبت شود:

```
ISSUE-YYYY-NNNN
```

نمونه قالب:

```
ISSUE-2026-0001
```

شناسه باید در گزارش تست، تصمیم، تغییر کد و گزارش توقف استفاده شود.

۱۵.۲. تعریف مسئله

هر مسئله باید شامل این فیلدها باشد:

- شناسه.
- شرح مشاهده‌شده.
- رفتار مورد انتظار.
- رفتار واقعی.
- روش بازتولید.
- شدت.

- اثر بر پذیرش.
- شواهد.
- فرضیه علت ریشه‌ای.
- تلاش‌های انجام‌شده.
- تصمیم بعدی.
- وضعیت.

تغییر عنوان یا شرح مسئله به‌تنهایی شمارش تلاش را صفر نمی‌کند.

توضیح داده شود. تقسیم ADR اگر دامنه مسئله واقعاً تغییر کند، باید رابطه آن با مسئله قبلی ثبت شود و دلیل ایجاد شناسه جدید در مسئله برای دورزدن سقف تلاش ممنوع است.

۱۵.۳. سقف سه تلاش

برای هر مسئله، حداکثر سه تلاش هدفمند مجاز است:

- تلاش اول: تشخیص علت و اصلاح مستقیم.
- تلاش دوم: بررسی فرضیه جایگزین و تغییر روش.
- تلاش سوم: بررسی مستقل و آخرین اصلاح محدود.

هر تلاش باید شواهد و نتیجه خود را داشته باشد.

اگر مسئله پس از سه تلاش حل نشد، کار روی همان مسیر متوقف می‌شود؛ مسئله به‌عنوان UNRESOLVED یا BLOCKED ثبت می‌شود و ادامه پروژه فقط در بخش‌های مستقل و مجاز ممکن است.

۱۵.۴. انواع توقف

توقف فوری یا توقف پذیرش در این موارد الزامی است:

- ابهام حقوقی مهم.
- خطر افشای کلید یا داده حساس.
- خرابی یا آلودگی داده.
- نشت اطلاعات.
- شکست آزمون حیاتی.
- شکست کنترل استقلال Oracle.
- ناسازگاری manifest یا امضا.
- تغییر غیرمجاز معیار پذیرش.
- نبود مرجع ضروری.
- پایان سه تلاش ناموفق.
- نبود مشخصات لازم برای نتیجه معتبر.
- درخواست یا اقدام خارج از محدوده مجاز.

توقف باید همراه با مدرک، شناسه مسئله و شرایط ازسرگیری ثبت شود.

---

۱۶. مدیریت دانش، قالب اسناد و انتقال بین عامل‌ها

.تمام اسناد اجرایی باید از قالب‌های استاندارد استفاده کنند. قالب‌های اختصاصی و ناسازگار بدون تصمیم ثبت‌شده مجاز نیستند

۱۶.۱. قالب‌های الزامی

:مسیر

`docs/templates/`

:حداقل قالب‌های زیر باید وجود داشته باشند

- `CURRENT_STATE.template.md`
- `OPEN_ISSUES.template.md`
- `ASSUMPTION_REGISTER.template.md`
- `DECISION_LOG.template.md`
- `NEXT_STAGE.template.md`
- `LAST_VERIFICATION.template.md`
- `MODULE_SUMMARY.template.md`
- `STAGE_REPORT.template.md`
- `HANDOFF.template.md`
- `ORACLE_SPEC.template.md`
- `EXPERIMENT_RECORD.template.md`
- `ACCEPTANCE_RECORD.template.md`

۱۶.۲. حداقل محتوای قالب‌ها

CURRENT_STATE

- .مرحله جاری
- .وضعیت پذیرش
- .آخرین شواهد معتبر
- .مشکلات باز
- .موانع
- .تغییرات اخیر
- .اقدام بعدی مجاز

OPEN_ISSUES

- .شناسه
- .شدت
- .شرح
- .روش بازتولید

- شواهد.
- تعداد تلاش.
- وضعیت.
- شرط بسته‌شدن.

ASSUMPTION_REGISTER

- شناسه فرض.
- شرح.
- علت فرض.
- دامنه اعتبار.
- سطح اطمینان.
- اثر در صورت نادرستی.
- روش آزمون.
- مالک بررسی.
- وضعیت.
- تصمیم مرتبط.

DECISION_LOG

- شناسه تصمیم.
- مسئله.
- گزینه‌ها.
- شواهد.
- ریسک‌ها.
- تصمیم نهایی.
- تصمیم‌گیر.
- تاریخ.
- اثر بر مراحل.
- شرایط بازنگری.

NEXT_STAGE

- هدف.
- پیش‌نیازها.
- ورودی‌ها.
- کارهای مجاز.
- کارهای ممنوع.
- تست‌ها.
- معیار PASS.
- معیار توقف.
- خروجی مورد انتظار.

LAST_VERIFICATION

- commit یا نسخه کد.

- نسخه محیط.
- فرمان‌های اجراشده.
- نتایج واقعی.
- تست‌های اجرانشده.
- هش‌ها.
- امضا.
- محدودیت‌ها.
- تصمیم پذیرش.

MODULE_SUMMARY

- هدف.
- API.
- وابستگی‌ها.
- تست‌ها.
- Oracle.
- محدودیت‌ها.
- وضعیت تأیید.
- مشکلات باز.

HANDOFF

- کار انجام‌شده.
- شواهد.
- تغییرات.
- تصمیم‌های مرتبط.
- مشکلات باز.
- تلاش‌های مصرف‌شده.
- گام بعدی.
- شروطی که عامل بعدی نباید نقض کند.

۱۶.۳. انتقال کار

عامل تحویل‌دهنده نباید فقط توضیح متنی ارائه دهد. باید شناسه artifactها، مسیر شواهد، نسخه کد، مسائل باز و شروط پذیرش را مشخص کند.

عامل دریافت‌کننده باید پیش از ادامه، صحت شواهد حیاتی را مستقلاً بررسی کند.

---

۱۷. حل اختلاف با SQ و مدیریت نسخه مرجع

۱۷.۱. دسته‌بندی اختلاف

هر اختلاف باید بررسی کند که منشأ آن یکی از موارد زیر است یا خیر:

- تفاوت نسخه SQ.
- تفاوت داده ورودی.
- تفاوت تنظیمات.
- تفاوت منطقه زمانی.
- تفاوت مدل هزینه.
- تفاوت زمان اجرای سفارش.
- تفاوت قواعد معامله.
- تفاوت محاسبات عددی.
- تفاوت تصادفی.
- نقص پیاده‌سازی مستقل.
- محدودیت شواهد.

۱۷.۲. تصمیم‌گیر

تصمیم نهایی درباره تعریف رفتار مرجع و پذیرش اختلاف باید در یک ADR ثبت شود.

عامل پیاده‌ساز نباید به‌تنهایی اختلافی را که روی معیار پذیرش همان قابلیت اثر دارد، نهایی کند.

باقی می‌ماند `BLOCKED` یا `UNKNOWN` بازبین مستقل باید شواهد را بررسی کند. در صورت اختلاف حل‌نشده، وضعیت قابلیت.

۱۷.۳. قاعده محافظه‌کاری

در حوزه اجرای معاملاتی، وقتی چند رفتار هنوز معتبرند و شواهد برای انتخاب قطعی کافی نیست، تا زمان حل اختلاف باید رفتار کم‌ریسک‌تر انتخاب شود؛ برای مثال:

- ورود دیرتر به‌جای ورود زودتر.
- حجم کمتر به‌جای حجم بیشتر.
- برآورد هزینه بالاتر به‌جای هزینه کمتر، در محدوده واقع‌بینانه و مستند.
- توقف به‌جای ادامه در صورت نامشخص‌بودن داده یا مشخصات سفارش.

این قاعده مجوز دستکاری محاسبات یا معرفی رفتار محافظه‌کارانه به‌عنوان رفتار واقعی SQ نیست. اختلاف باید همچنان ثبت شود.

۱۷.۴. نسخه مرجع

پیاده‌سازی سازگار باید رفتار آخرین نسخه مرجع تأییدشده را دنبال کند.

تغییر نسخه مرجع نیازمند ADR است.

فایل زیر برای ثبت نسخه‌ها در نظر گرفته می‌شود:

`docs/behavior_registry.yaml`

هر رکورد باید حداقل شامل شناسه قابلیت، نسخه مرجع، وضعیت پشتیبانی، تاریخ تأیید، شواهد، محدودیت‌ها و `deprecated_after` باشد.

نسخه قدیمی فقط در صورت نیاز مستند و با آزمون جداگانه حفظ می‌شود. رفتار نسخه‌های مختلف نباید بدون تفکیک در یک نتیجه پژوهشی ادغام شود.

---

۱۸. موفقیت پروژه و معیارهای پژوهشی

موفقیت پروژه در سه سطح مستقل سنجیده می‌شود.

۱۸.۱. موفقیت فنی

پیش‌فرض پذیرش:

- صفر خطای بحرانی باز.
- صددرصد تست‌های Critical Path موفق.
- تکمیل artifactهای اجباری.
- هش و امضای معتبر برای artifactهای حیاتی.
- عبور از آزمون‌های leakage و بازتولیدپذیری.
- نبود تغییر غیرمجاز در معیارها.
- تأیید مستقل نتایج پذیرش.

ماژول‌های حیاتی نیز باید به آستانه مصوب برسد Mutation Score.

۱۸.۲. موفقیت پژوهشی

موفقیت پژوهشی به معنی اثبات سودآوری قطعی نیست.

یک برنامه ارزیابی اولیه می‌تواند شامل حداقل سه استراتژی دارای حداقل صد معامله معتبر برای هر استراتژی باشد؛ اما این تعداد به‌تنهایی برای نتیجه‌گیری آماری کافی نیست.

معیارهای ارزیابی، سطح اطمینان، اندازه نمونه و محدودیت‌های انتخاب باید پیش از اجرای آزمایش ثبت شوند.

هدف اولیه پژوهش، اثبات فرایند معتبر و جلوگیری از نتایج کاذب است. اگر هیچ استراتژی از آزمون خارج از نمونه عبور نکند، نتیجه پژوهش ممکن است از نظر فنی معتبر و از نظر وجود مزیت معاملاتی منفی باشد.

۱۸.۳. موفقیت رقابتی

فقط زمانی مجاز است که SQ ادعای نزدیکی به:

- قابلیت و نسخه مرجع مشخص باشد.
- Oracle مستقل موجود باشد.
- معیار مقایسه پیش از آزمون ثبت شده باشد.
- تلورانس مشخص باشد.
- مجموعه آزمون مستقل باشد.
- نتیجه اختلاف‌ها گزارش شده باشد.

بالاتر از یک یا هر آستانه منفرد دیگر، به‌تنهایی موفقیت رقابتی یا سودآوری آینده را اثبات نمی‌کند Sharpe یک استراتژی با.

۱۸.۴. اصلاح چندبارآزمایی

پیش از جست‌وجوی گسترده، معیار اصلی انتخاب استراتژی باید ثبت شود.

برای مجموعه نتایج منتخب، روش مناسب کنترل چندبارآزمایی باید انتخاب و گزارش شود. گزینه‌ها می‌توانند شامل Deflated Sharpe Ratio، اصلاح Bonferroni یا روش آماری مناسب دیگر باشند.

روش نهایی باید با ساختار آزمایش و وابستگی میان آزمون‌ها سازگار باشد؛ انتخاب روش صرفاً برای بهتر نشان‌دادن نتیجه ممنوع است.

---

۱۹. MVP واقعی و مراحل توسعه

۱۹.۱. MVP-1: مسیر حداقلی قابل آزمون

MVP اولیه فقط شامل این شش قابلیت است:

۱. دریافت یک فایل CSV با schema از پیش تعریف‌شده.

۲. ممیزی پایه شامل ستون‌ها، OHLC و ترتیب زمانی.

۳. اجرای یک استراتژی قاعده‌محور ساده، مانند تقاطع دو میانگین متحرک.

۴. بک‌تست ساده بدون هزینه، با برچسب صریح `NO_COST_RESEARCH_ONLY`.

۵. خروجی CSV معاملات.

۶. یک آزمون end-to-end که ورودی مشخص را به خروجی قابل بررسی تبدیل کند.

MVP-1 قرار نیست قابلیت نهایی پروژه را پوشش دهد. هدف آن اثبات صحت مسیر حداقلی است.

۱۹.۲. MVP-2

پس از پذیرش MVP-1:

- مدل هزینه.
- گزارش قابل بررسی.
- رجیستری آزمایش.
- مدیریت نسخه داده.
- امضای artifactهای حساس.
- پایش خودکار.
- گسترش ممیزی داده.

۱۹.۳. توسعه بعدی

رابط کاربری و اتصال MT5، تکامل در مراحل مستقل پیاده‌سازی می‌شوند. چند فایل و تایم‌فریم، تولید استراتژی، Monte Carlo، Walk-Forward، PyTorch، ML کوچک، محدوده کل پروژه را کوچک نمی‌کند؛ فقط ریسک ساخت اولیه را کاهش می‌دهد MVP.

---

۲۰. عامل‌ها، مسئولیت‌ها و محدودیت‌ها

نقش‌ها باید بر اساس وظیفه و مجوز تعریف شوند، نه بر اساس عنوان نمایشی.

نقش‌های پایه:

- **Planner:** تقسیم کار و کنترل وابستگی‌ها.
- **Implementer:** تغییر محدود کد در محدوده مصوب.
- **Oracle/Behavior Explorer:** کشف و ثبت رفتار مرجع.
- **Test Engineer:** طراحی و اجرای آزمون‌ها.
- **Independent Auditor:** بررسی مستقل شواهد و نتیجه.
- **Security Reviewer:** بررسی امنیت، اسرار و مرزهای دسترسی.
- **Research Analyst:** تحلیل آماری و روش پژوهش.
- **Knowledge Keeper:** حفظ مستندات، تصمیم‌ها و انتقال زمینه.
- **Release Approver:** تأیید نهایی انتشار.

یک عامل می‌تواند چند نقش را اجرا کند، اما استقلال منطقی آزمون نباید از بین برود.

اگر یک عامل کد تولیدی را نوشته باشد، صرف اجرای تستی که خودش طراحی کرده است، استقلال کامل را ثابت نمی‌کند. آزمون Oracle مستقل باید از یا روش مرجع جداگانه استفاده کند.

۲۰.۱. محدودیت اختیار

عامل‌ها مجاز نیستند:

- دامنه پروژه را بی‌اجازه تغییر دهند.
- معیار پذیرش را پس از مشاهده نتیجه دست‌کاری کنند.
- داده یا کلید محرمانه را منتشر کنند.
- آزمون را بدون ثبت دلیل حذف کنند.
- وضعیت `UNKNOWN` را به `PASS` تبدیل کنند.
- به‌جای کاربر مجوز اجرای واقعی صادر کنند.
- به مخزن یا سرویس خارج از محدوده مجاز تغییر دهند.

۲۰.۲. حل اختلاف عامل‌ها

اختلاف باید به شکل مسئله ثبت‌شده، همراه با شواهد، گزینه‌ها و ریسک‌ها ارائه شود.

تصمیم‌های مربوط به امنیت، اعتبار آماری، رفتار مرجع یا تغییر معیار پذیرش به بازبینی مستقل نیاز دارند.

است؛ نه رأی‌گیری بر اساس تعداد عامل‌ها BLOCKED اگر اختلاف با شواهد حل نشود، نتیجه.

---

۲۱. همزمانی، ادغام و کنترل تغییرات

پس از تأیید شروع کار روی مخزن، سیاست پیشنهادی چنین است:

- نام‌گذاری شاخه‌ها: `stage/NN-name`.
- تغییرات هر مرحله محدود به محدوده همان مرحله.
- ممنوعیت ویرایش همزمان یک فایل حساس بدون هماهنگی.
- قبل از ادغام diff الزام بررسی.
- الزام بازبینی مستقل برای مسیرهای حیاتی.
- اجرای کنترل‌های خودکار پیش از ادغام.
- ثبت تصمیم‌های معماری در ADR.
- ممنوعیت ادغام تغییرات دارای تست حیاتی شکست‌خورده.
- ممنوعیت انتشار artifact بدون بررسی امنیت و اسرار.

پس از بررسی محیط branch protection و سیاست pull request قفل فایل، بررسی pre-commit، ابزارهای اجرایی مانند واقعی انتخاب می‌شوند. نام‌بردن از ابزار به معنی نصب یا فعال‌بودن آن نیست.

باقی می‌ماند PROPOSED تا زمان اجرای واقعی این سیاست‌ها، وضعیت آن‌ها.

---

۲۲. پایش عامل‌ها و تحلیل خطا

پایش باید خودکار و قابل بررسی باشد؛ گزارش دستی به‌تنهایی کافی نیست.

۲۲.۱. شاخص‌ها

- نرخ موفقیت تست‌های حیاتی.
- تعداد خطاهای بحرانی باز.
- تعداد تلاش‌های مصرف‌شده برای هر مسئله.
- تعداد تغییرات بدون تست مرتبط.
- تعداد تست‌های حذف یا غیرفعال‌شده.
- Mutation Score در ماژول‌های تحت پوشش.
- نرخ رد بازبینی.
- تعداد artifact های بدون امضا.
- تعداد نتایج غیرقابل بازتولید.
- تعداد اختلاف‌های حل‌نشده با Oracle.
- تعداد فرض‌های تأییدنشده.

- تعداد تغییرات خارج از محدوده.
- زمان و تعداد دفعات توقف.

۲۲.۲. اجرای پایش

باید در هر دروازه پذیرش اجرا شود `monitor_agents.py`.

گزارش کند، نه صفر `UNKNOWN` اسکریپت باید منبع هر شاخص را مشخص کند. اگر اطلاعاتی در دسترس نباشد، باید.

۲۲.۳. تحلیل الگوی خطا

خطاهای تکرارشونده باید به‌صورت دسته‌بندی‌شده بررسی شوند:

- خطای فهم مسئله.
- خطای پیاده‌سازی.
- خطای تست.
- خطای محیط.
- خطای داده.
- خطای Oracle.
- خطای مدیریت زمینه.
- خطای تغییر غیرمجاز.
- خطای گزارش‌دهی.

اگر یک الگو تکرار شود، اصلاح باید در فرایند یا کنترل خودکار انجام شود؛ نه فقط در همان فایل.

---

۲۳. امنیت، داده و انتشار عمومی

۲۳.۱. اسرار

گذرواژه‌ها، توکن‌ها، اطلاعات حساب و کلید امضا نباید وارد مخزن عمومی شوند، API کلیدهای.

۲۳.۲. داده بازار

پیش از انتشار داده‌ها باید مجوز توزیع و شرایط منبع بررسی شود. دسترسی کاربر به داده، لزوماً به معنی مجازبودن انتشار عمومی آن نیست.

۲۳.۳. دارایی‌های اختصاصی

نباید بدون مجوز وارد پروژه یا مخزن عمومی شوند SQ فایل‌ها، کدها، داده‌ها یا دارایی‌های اختصاصی.

۲۳.۴. کنترل انتشار

پیش از انتشار عمومی باید این موارد بررسی شوند:

- اسرار و فایل‌های محیطی.
- داده‌های خصوصی یا دارای محدودیت انتشار.
- گزارش‌های حاوی اطلاعات حساس.
- مجوز وابستگی‌ها.
- مجوز داده‌ها.
- اسناد حقوقی مرتبط.
- تست‌های حیاتی.
- وضعیت خطاهای باز.
- امضا و هش artifactهای انتشار.

---

۲۴. دروازه‌های پذیرش

هر مرحله باید دارای `PASS`، `FAIL` یا `BLOCKED` باشد.

PASS

فقط زمانی مجاز است که:

- پیش‌نیازها برآورده شده باشند.
- معیارهای پذیرش از پیش تعریف شده باشند.
- آزمون‌های لازم واقعاً اجرا شده باشند.
- خروجی‌ها قابل بررسی باشند.
- آزمون مستقل لازم موفق شده باشد.
- artifactها ثبت شده باشند.
- مشکلات بحرانی باز وجود نداشته باشند.
- گزارش مرحله تکمیل شده باشد.

FAIL

زمانی ثبت می‌شود که آزمون اجرا شده و معیار پذیرش را نقض کرده باشد.

BLOCKED

زمانی ثبت می‌شود که آزمون به‌دلیل نبود داده، ابزار، مجوز، Oracle، دسترسی یا پیش‌نیاز ضروری قابل تکمیل نباشد.

`BLOCKED` به معنی موفقیت نیست.

۲۴.۱. بسته تحویل هر مرحله

هر مرحله باید حداقل این موارد را داشته باشد:

- `00_SCOPE.md`
- `01_PLAN.md`
- `02_IMPLEMENTATION.md`
- `03_TEST_RESULTS.md`
- `04_AUDIT.md`
- `05_DECISIONS.md`
- `06_HANDOFF.md`
- `evidence/`

برای مراحل ساده، اسناد می‌توانند کوتاه باشند، اما حذف اطلاعات ضروری مجاز نیست.

---

۲۵. دروازه نهایی اجرای معاملاتی

مسیر اجرای واقعی باید از پژوهش مستقل باشد و حداقل این مراحل را طی کند:

۱. پذیرش فنی موتور.

۲. پذیرش صحت داده و مدل هزینه.

۳. پذیرش آماری استراتژی.

۴. آزمون خارج از نمونه و پایداری.

۵. آزمون اجرای آزمایشی و دمو.

۶. بررسی امنیت، محدودیت ریسک و توقف اضطراری.

۷. مجوز صریح کاربر برای فعال‌سازی اجرای واقعی.

هیچ عامل یا ماژولی مجاز نیست صرفاً به دلیل نتیجه مثبت بک‌تست، سفارش واقعی ارسال کند.

فعال‌سازی واقعی باید با تنظیمات محدودکننده، ثبت رویدادها و امکان توقف فوری همراه باشد.

بخش سوم — پروتکل اجرا و تحویل

۲۶. ترتیب شروع هر کار

عامل مسئول باید:

۱. وضعیت واقعی پروژه را از فایل‌ها و شواهد موجود بخواند.

۲. آخرین مرحله تأییدشده را تعیین کند.

۳. مشکلات باز و محدودیت‌ها را بررسی کند.

۴. وابستگی‌ها و پیش‌نیازهای مرحله بعد را بررسی کند.

۵. دامنه کار و معیار پذیرش را مشخص کند.

۶. Oracle و تست‌های مستقل را تعریف کند.

۷. تغییرات محدود را اجرا کند.

۸. تست‌ها و کنترل‌های لازم را اجرا کند.

۹. نتیجه را مستقلاً بررسی کند.

۱۰. گزارش، شواهد و تحویل مرحله را ثبت کند.

۱۱. مرحله بعد را آغاز کند، PASS فقط در صورت.

اگر مرحله FAIL یا BLOCKED به مشروط باشد، عبور به مرحله وابسته ممنوع است. کار مستقل و بی‌خطر می‌تواند ادامه یابد، اینکه وضعیت توقف یا وابستگی را دور نزند.

۲۷. قوانین اجرای فرمان و گزارش نتیجه

هر فرمانی که نتیجه‌اش مبنای پذیرش است باید واقعاً اجرا شود.

گزارش باید شامل فرمان، محیط، خروجی، کد خروج، نسخه کد و محدودیت‌های مشاهده‌شده باشد.

اگر اجرا ممکن نباشد، باید دلیل آن ثبت شود.

هیچ نتیجه‌ای نباید صرفاً از روی نام تست، کد موجود یا گزارش تاریخی به‌عنوان وضعیت فعلی تأیید شود.

فقط سابقه تاریخی‌اند و برای پذیرش compileall، unittest یا pytest، گزارش‌های قدیمی، از جمله گزارش‌های قبلی موفقیت نسخه فعلی باید در محیط فعلی دوباره اجرا شوند.

۲۸. بازبینی پنج‌مرحله‌ای

هر سند مهم، طرح معماری یا تغییر حیاتی باید از پنج بازبینی عبور کند:

بازبینی اول — کامل‌بودن: آیا هدف، نیازها و قیود پوشش داده شده‌اند؟

بازبینی دوم — تناقض: آیا بخش‌ها با یکدیگر تعارض دارند؟

بازبینی سوم — اجراپذیری: آیا هر الزام به آزمون، ابزار، مدرک یا تصمیم مشخص متصل است؟

بازبینی چهارم — امنیت و استقلال: آیا راهی برای تقلب در تست، نشت اطلاعات یا تأیید دوری وجود دارد؟

بازبینی پنجم — سادگی و انتقال: آیا سند بدون تکرار غیرضروری، برای عامل جدید قابل فهم و اجراست؟

.این پنج بازبینی، بررسی کیفیت سند هستند؛ جایگزین آزمون واقعی نرم‌افزار نمی‌شوند

۲۹. وضعیت فعلی و کارهای تأییدنشده

:موارد زیر تا زمان بررسی واقعی باید باز بمانند

- .وضعیت فعلی فایل‌ها و کد پروژه
- .دسترسی واقعی به پوشه محلی
- .دسترسی و وضعیت مخزن GitHub
- .وضعیت فعلی تست‌ها
- .صحت و دسترس‌پذیری دیتاست اصلی
- .رفتار دقیق نسخه مرجع SQ
- .مشخصات مستند و جاری نمادها و هزینه‌های Aron Group
- .تصمیم حقوقی مستند درباره روش مهندسی معکوس
- .انتخاب ابزارهای نهایی
- .پیاده‌سازی و آزمون `monitor_agents.py`
- .پیاده‌سازی و آزمون سازوکار امضا
- .تصویب آستانه‌های متناسب با کد واقعی
- .پیاده‌سازی MVP-1
- .تأیید هرگونه ادعای عملکرد معاملاتی

.هیچ‌یک از این موارد صرفاً با درج در این سند انجام‌شده محسوب نمی‌شود

۳۰. شرایط تصویب و تغییر سند

.این سند مرجع الزامات پروژه است. تغییر آن باید در `DECISION_LOG.md` یا ADR ثبت شود

:هر تغییر باید مشخص کند

- .چه چیزی تغییر می‌کند
- .چرا تغییر لازم است
- .کدام الزام قبلی را متأثر می‌کند
- .اثر آن بر معماری و آزمون‌ها چیست
- .آیا معیار پذیرش تغییر می‌کند
- .چه کسی تغییر را تأیید کرده است
- .چه آزمون‌هایی باید دوباره اجرا شوند

.تغییر معیار پس از مشاهده نتایج، بدون ثبت تصمیم و اجرای دوباره ارزیابی، مجاز نیست

اگر سند مادر و ضمیمه اجرایی تعارض داشته باشند، اصل حفظ هدف کلان و قابلیت‌های پروژه حاکم است؛ سپس تعارض باید ثبت و دامنه نهایی پروژه را حذف کند، MVP حل شود. ضمیمه اجرایی مجاز نیست با کوچک‌کردن ADR از طریق.

۳۱. معیار نهایی پذیرش پروژه

پروژه فقط زمانی از نظر فنی آماده انتشار تلقی می‌شود که:

- قابلیت‌های ادعاشده با شواهد معتبر همراه باشند.
- تست‌های حیاتی و کنترل‌های امنیتی موفق باشند.
- فرایند داده تا خروجی قابل بازتولید باشد.
- نتیجه آزمون مستقل بررسی شده باشد.
- مسائل بحرانی باز وجود نداشته باشند.
- مستندات و قالب‌های دانش کامل باشند.
- وضعیت مجوز کد، داده و وابستگی‌ها بررسی شده باشد.
- اسرار از بسته انتشار حذف شده باشند.
- نصب و اجرای بسته در محیط تمیز آزمایش شده باشد.
- انتقال به مخزن هدف با اجازه کاربر انجام شود.

پروژه فقط زمانی از نظر پژوهشی موفق تلقی می‌شود که روش ارزیابی معتبر باشد و نتایج، از جمله نتایج منفی، به‌صورت شفاف گزارش شوند.

مستقل و معیارهای از پیش Oracle موفق تلقی می‌شود که قابلیت‌های ادعاشده با SQ پروژه فقط زمانی از نظر سازگاری با تعیین‌شده ارزیابی شده باشند.

هیچ‌یک از این سه نوع موفقیت، سودآوری آینده را تضمین نمی‌کند.

بخش چهارم — دستور اجرایی برای عامل بعدی

هنگام ادامه کار روی Strategy Quant:

۱. ابتدا وضعیت واقعی پروژه و شواهد موجود را بررسی کن.

۲. هیچ تغییری در GitHub یا مخزن بیرونی بدون اجازه صریح کاربر انجام نده.

۳. سند مادر را حفظ کن و قابلیت‌های اصلی آن را حذف نکن.

۴. ابتدا دروازه حقوقی، محدوده دسترسی و وضعیت فعلی پروژه را روشن کن.

۵. تست مستقل و معیار پذیرش آن را تعریف کن، Oracle، پیش از پیاده‌سازی هر قابلیت مهم.

۶. MVP-1 را به شش قابلیت تعیین‌شده محدود نگه دار.

۷. مشکلات را با شناسه یکتا ثبت کن و سقف سه تلاش را دور نزن.

۸. هر الزام اجرایی را به تست، اسکریپت، مدرک یا تصمیم ثبت‌شده متصل کن.

۹. هیچ آزمونی را بدون اجرای واقعی موفق اعلام نکن.

۱۰. در پایان هر مرحله، گزارش، شواهد، وضعیت پذیرش و تحویل استاندارد را ثبت کن.

۱۱. مانع را مشخص کن و مسیر وابسته را متوقف نگه دار، BLOCKED یا FAIL به مرحله بعد برو؛ پس از PASS،

۱۲. هیچ اتصال یا سفارش واقعی را بدون عبور از دروازه مستقل و مجوز صریح کاربر فعال نکن.

هدف نهایی: ساخت یک پلتفرم مستقل و قابل ممیزی که داده را به پژوهش، پژوهش را به استراتژی قابل ارزیابی و نتایج را به شواهد قابل بازبینی تبدیل کند؛ بدون اینکه خروجی نامعتبر، خطای پنهان یا ادعای اثبات‌نشده به‌عنوان موفقیت پذیرفته شود.